COMPSCI 5100 ML & AI for Data Science

Fani Deligianni

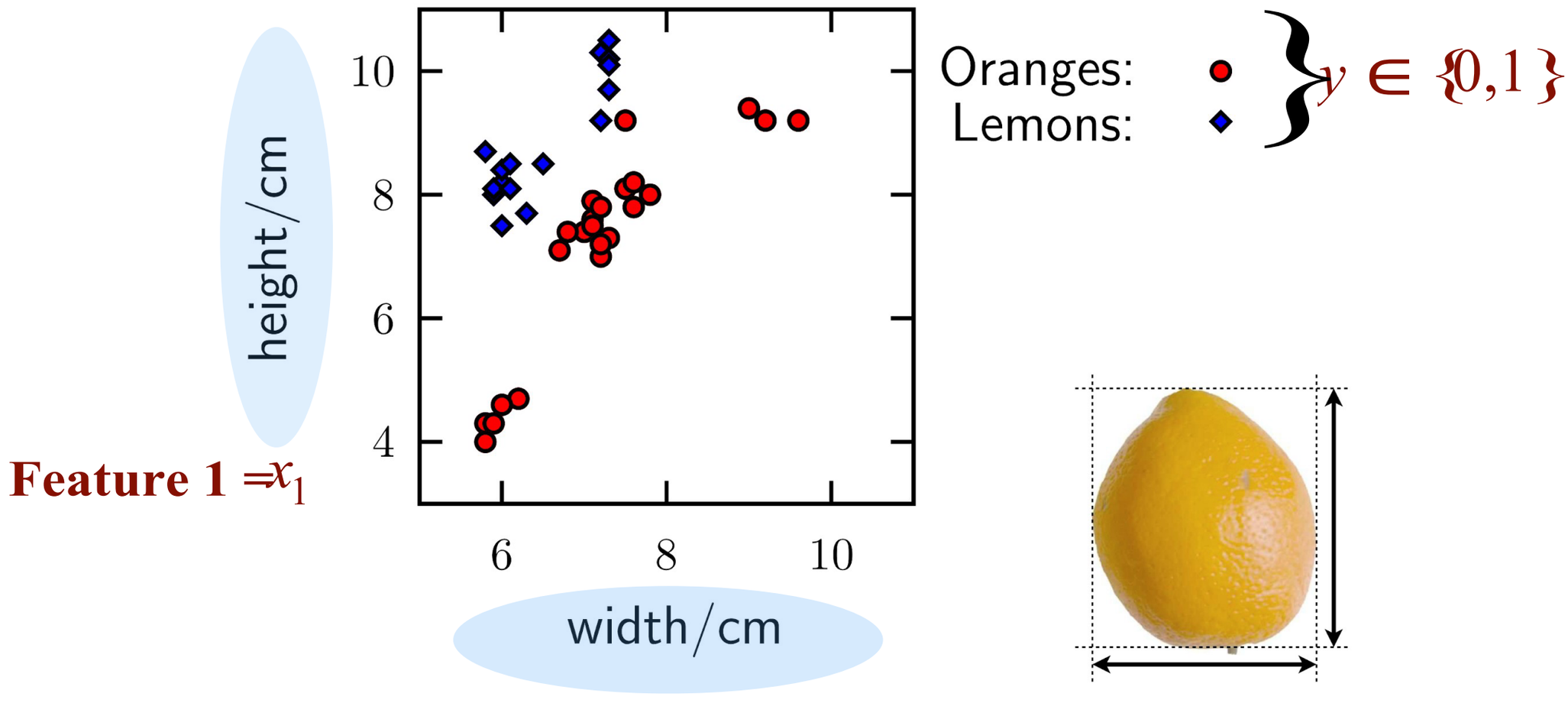
Fani.deligianni@glasgow.ac.uk

Classification: Logistic Regression

Some slides are adapted from previous lectures: Dr. Yuan, Dr. Guha, Dr. Gooya

Logistic regression

- A popular statistical model used for **binary** classification
- Can be derived from **Linear Regression**



Least squares for classification

1. **Training Data & Features**
   - **Data points:** $(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)$ where $\boldsymbol{x_i} = [x_{i1}, x_{i2}]^T$
   - **Target labels:** $t_i \in \{-1,1\}$
2. **Decision Boundary & Linear Model**
   - Classification is performed by finding a hyper-plane (decision boundary) that separates the two classes.
   - The decision boundary is given by y(x) = 0, where y(x) is the model's prediction (discriminant function)
   - For instance, using a linear model:

$$y(x_i) = w_0 + w_1 x_{i1} + w_2 x_{i2} = w_0 + w^T x_i$$

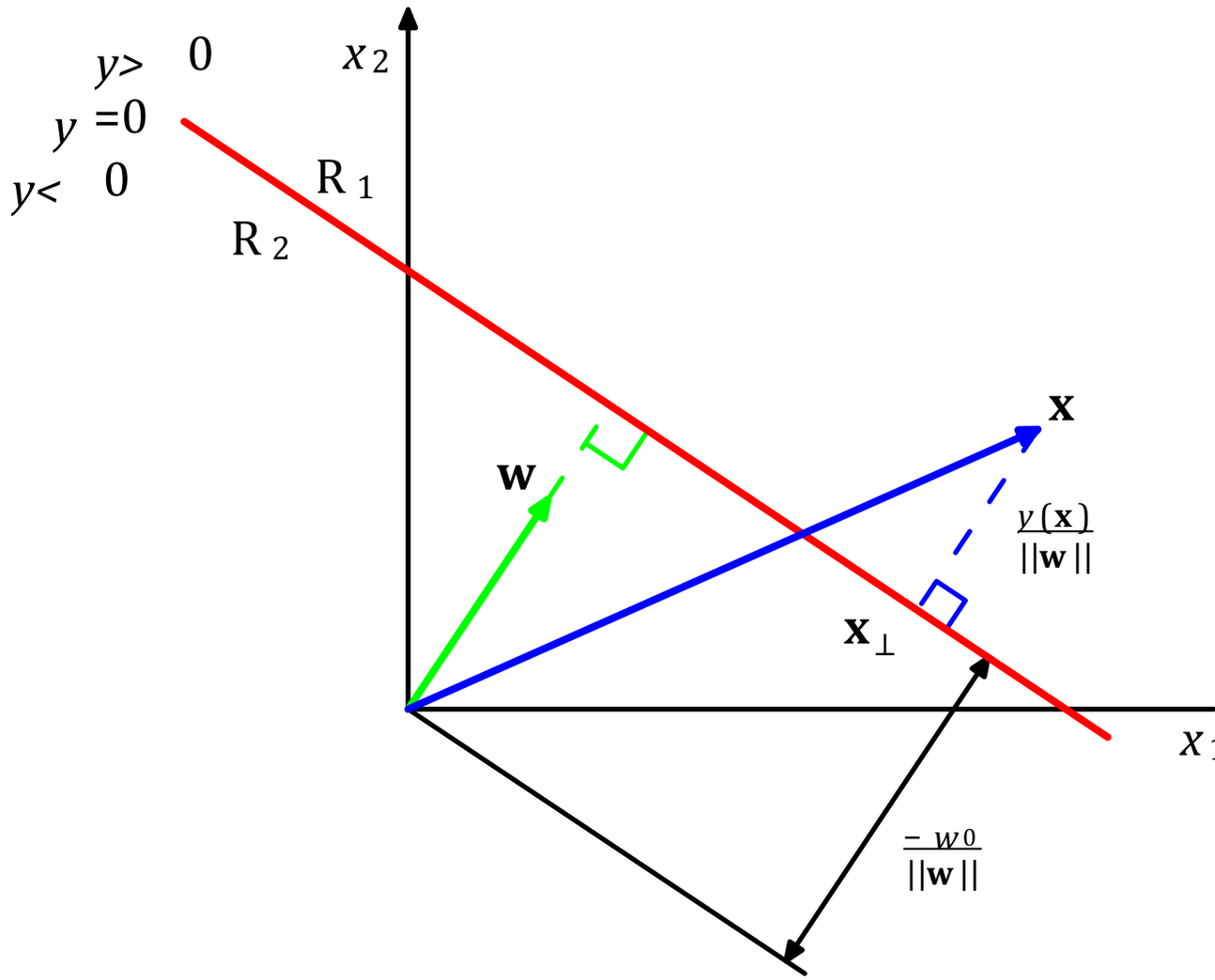
   (Note: In this 2D feature space, y(x) = 0 represents a straight line.)
3. **Cost Function & Optimisation**
   a. The parameters *w* can be obtained by minimizing the sum of squared errors cost function:

$$J(w) = \sum_{i=1}^{N} (y_i - t_i)^2, \text{ with: } w^* = \arg\min_w J(w)$$

Geometry of the linear regression

- Distance of $\mathbf{x}$ from decision boundary $y = 0$ is proportional to $y(\mathbf{x})$
- $\mathbf{w}$ is normal to the decision boundary.
- Hence, the cost function penalises the distance of points from the decision plane

Figure 4.1 Illustration of the geometry of a linear discriminant function in two dimensions. The decision surface, shown in red, is perpendicular to $\mathbf{w}$, and its displacement from the origin is controlled by the bias parameter w_0. Also, the signed orthogonal distance of a general point $\mathbf{x}$ from the decision surface is given by $y(\mathbf{x})/||\mathbf{w}||$.



Why bother with logistic regression?

- Classification based on least-squares is prone to outliers.
- See (C. Bishop's PRML Book, Ch. 4)

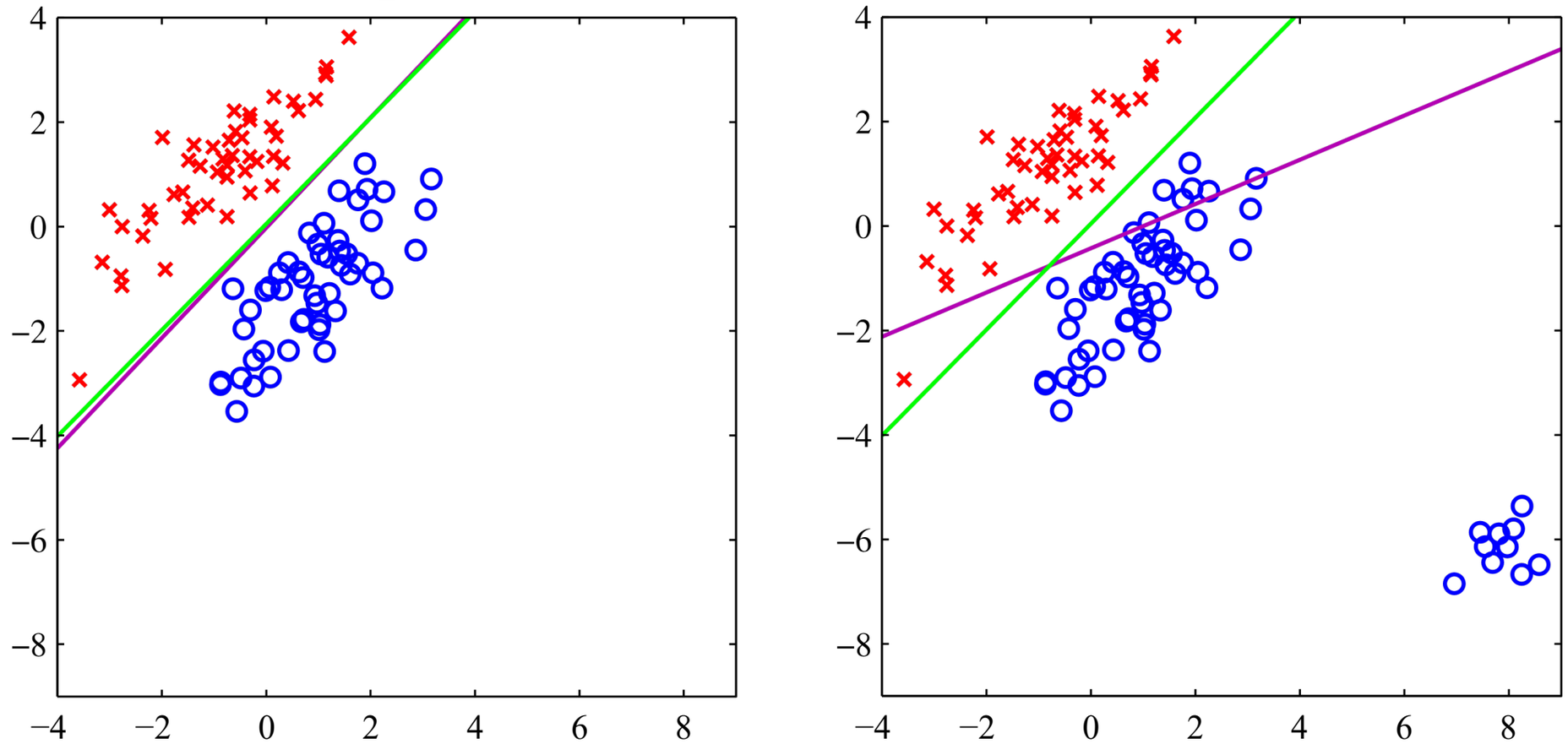


Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

Logistic regression

- **Target Variable:** Without loss of generality, assume binary labels $t \in \{0,1\}$
- **The Solution to outliers:** Pass the linear model through a non-linear activation function to suppress extreme values:

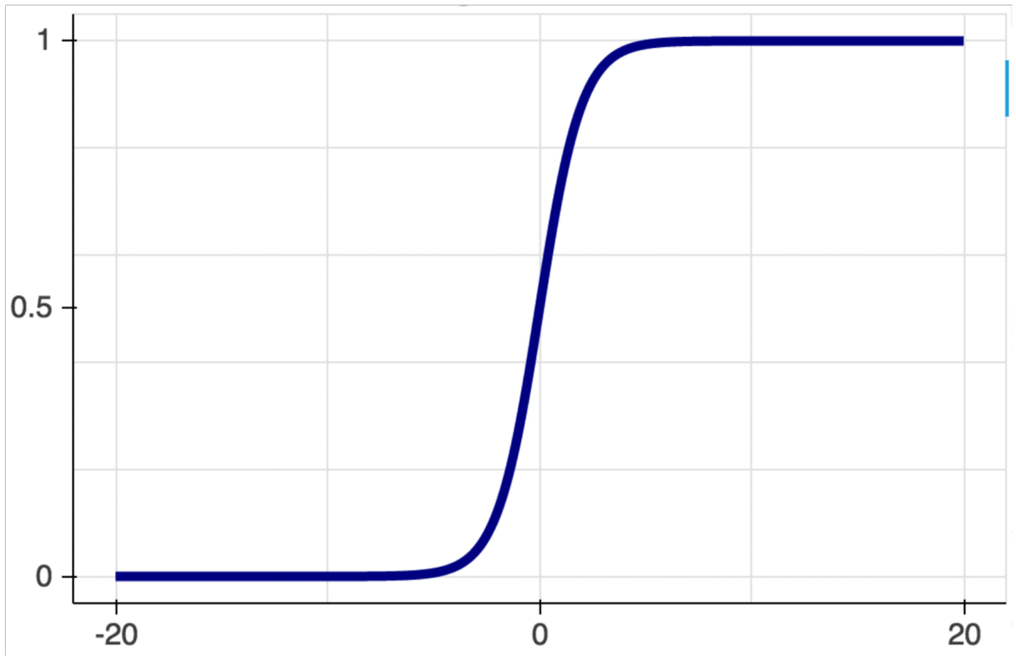
$$y(x) = \sigma(w_0 + w^T x)$$

- **The Logistic (Sigmoid) Function:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **The Bias Trick**
  - **Absorb the bias:** $\boldsymbol{w} \leftarrow [w_0, w]^T$
  - **Augment the input** $\boldsymbol{x} \leftarrow [1, x]^T$
- **Compact Model Form**

$$y(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T \boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{w}^T \boldsymbol{x}}}$$



Logistic regression

- In standard linear regression or binary classification, the raw linear output z:

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_D x_D$$

- In vector notation, if we define our weight vector as $\mathbf{w} = [w_1, w_2, \ldots, w_D]^T$ and our feature vector as $\mathbf{x} = [x_1, x_2, \ldots, x_D]^T$, we can express this as:
- Define the Augmented Vectors:

Augmented Weight Vector:	**Augmented Input Vector:**
$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_D \end{bmatrix}$	$x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix}$

- Compute the Dot Product of the Augmented Vectors:

$$w^T x = (w_0 \cdot 1) + (w_1 x_1 + w_2 x_2 + \cdots + w_D x_D) = w_0 + \mathbf{w}^T \mathbf{x}$$

- Plug into the Sigmoid Function:

$$y(x) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

Logistic regression: Probabilistic Interpretation

- **Posterior Class Probabilities:** Logistic regression outputs can be interpreted directly as probabilities:

$$y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}} = P(t = 1 \mid \mathbf{x}, \mathbf{w})$$

$$P(t = 0 \mid \mathbf{x}, \mathbf{w}) = 1 - P(t = 1 \mid \mathbf{x}, \mathbf{w})$$

- **Parameter Estimation:** Learn the optimal weight parameters ***w***
- **Optimisation Goal:** We need a well-defined cost function ***J(w)*** over which we can perform optimization.

Logistic Regression: Cost function

- **Individual Class Likelihoods:** For a binary label $t_i \in \{0,1\}$, the probabilities are:

$$P(t_i = 1 \mid \mathbf{x}_i, \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x}_i) \,,$$
$$P(t_i = 0 \mid \mathbf{x}_i, \mathbf{w}) = 1 - \sigma(\mathbf{w}^T \mathbf{x}_i)$$

- **Combined Probability Expression (Bernoulli Trick):**

$$p(t_i \mid \mathbf{x}_i, \mathbf{w}) = [\sigma(\mathbf{w}^T \mathbf{x}_i)]^{t_i} [1 - \sigma(\mathbf{w}^T \mathbf{x}_i)]^{1-t_i} = y_i^{t_i} (1 - y_i)^{1-t_i}$$

- **Log-Likelihood Function:**

$$\log p\,(t_i \mid \mathbf{x}_i, \mathbf{w}) = t_i \log[\sigma(\mathbf{w}^T \mathbf{x}_i)] + (1 - t_i) \log[1 - \sigma(\mathbf{w}^T \mathbf{x}_i)]$$

Logistic Regression: Optimisation

- **Optimization Objective:** We find the optimal weights $\mathbf{w}^*$ by minimizing the negative log-likelihood (which is mathematically equivalent to maximizing the log-likelihood).

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w})$$

- **Cost Function ($J(\mathbf{w})$):** Summed across all training samples, the negative log-likelihood loss is:

$$J(\mathbf{w}) = -\sum_{i=1}^{N} \log p\,(t_i \mid \mathbf{x}_i, \mathbf{w})$$

- L_2 **Regularization:** To prevent overfitting when feature spaces are complex, we can add an L_2 penalty term ($\lambda\|\mathbf{w}\|^2$):

$$J(\mathbf{w}) = -\sum_{i=1}^{N} \log p\,(t_i \mid \mathbf{x}_i, \mathbf{w}) + \lambda\|\mathbf{w}\|^2$$

Logistic regression: From Probabilities to Discrete Labels

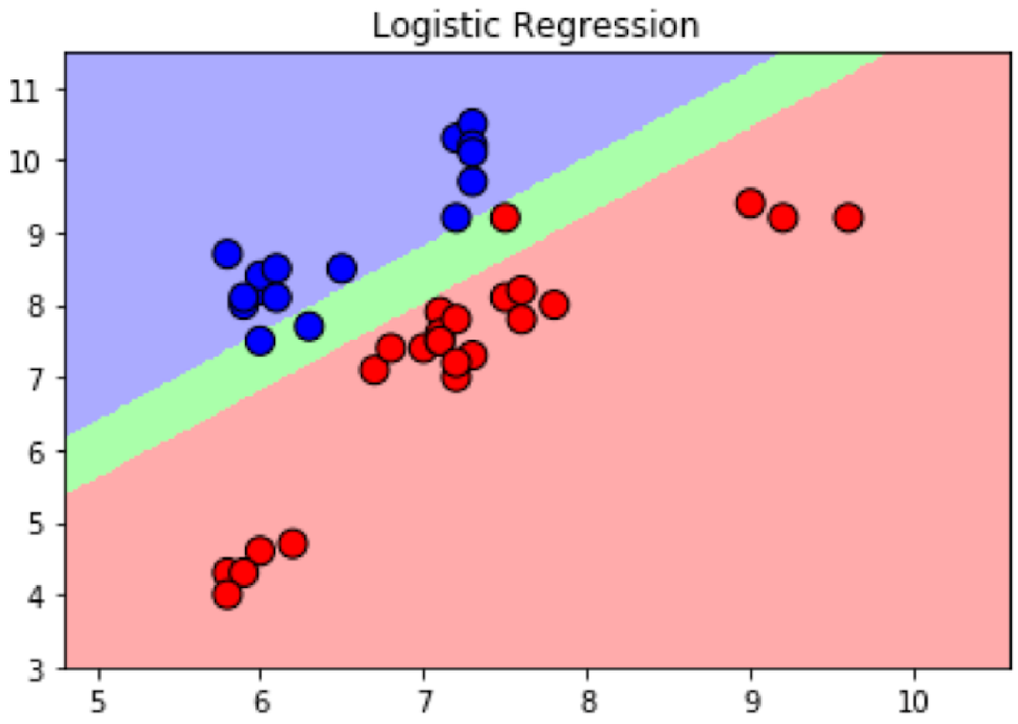
- **Converting Prob Scores:** Logistic regression outputs probabilities. To make a final decision, we must map them to discrete class labels.
- **Default Decision Rule:** Assign the label of whichever class has the higher probability.
- **Custom Decision Thresholds:** We can adjust the threshold based on application requirements:

$$\hat{y}_{\text{new}} = 1 \text{ if } P(y_{\text{new}} = 1 \mid \mathbf{x}_{\text{new}}, \mathbf{w}) \geq 0.6$$

- **The "Reject" Option:** We can choose to withhold a prediction when the probability difference between classes is too small to ensure confidence.
- **Threshold Optimization:** Thresholds can be optimized using cross-validation (CV), though they remain highly data-dependent.

In [59]:
```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(X, t)
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=cmap_light)
# Plot also the training points
plt.scatter(X[:, 0], X[:, 1], c=t, cmap=cmap_bold,
            edgecolor='k', s=100)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.title("Logistic Regression")
mean_cv_score = np.mean( cross_val_score(clf, X, t, cv=5) )
print("5-fold averae CV error:", 1-mean_cv_score)
```

```
5-fold averae CV error: 0.025000000000000022
```



Example: orange and lemon data

Logistic regression summary

Simple

- Linear model passed through a non-linear function
- Probabilistic interpretation
- Robust to outliers
- Smooth cost function
- Learnable params = feature weights

…but

- Assumes linearity between dependent and independent variables.
- If the number of samples< feature dimension, we can have overfitting
- Too simple for complex data

Classification: Performance evaluation

Performance evaluation

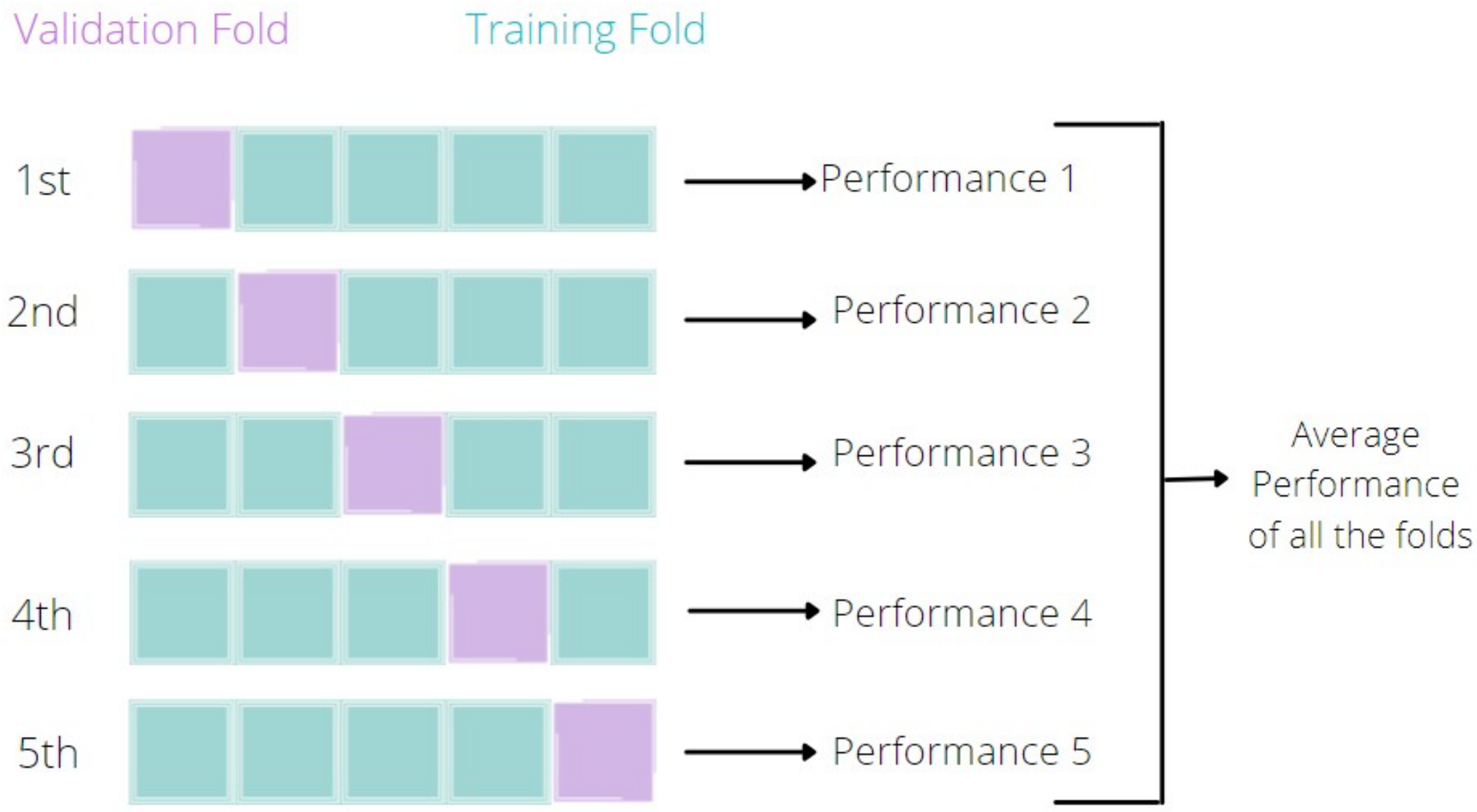
- **Evaluation strategy**
- How to split data for training and testing

- **Evaluation metrics**
- How to measure performance accurately

- **Benchmarking**
- Compare results against other ‘known’ results

image from: pub.towardsai.net

- **Cross validation**



- **Leave one subject out**
- Particularly useful for classification tasks involving human centric data

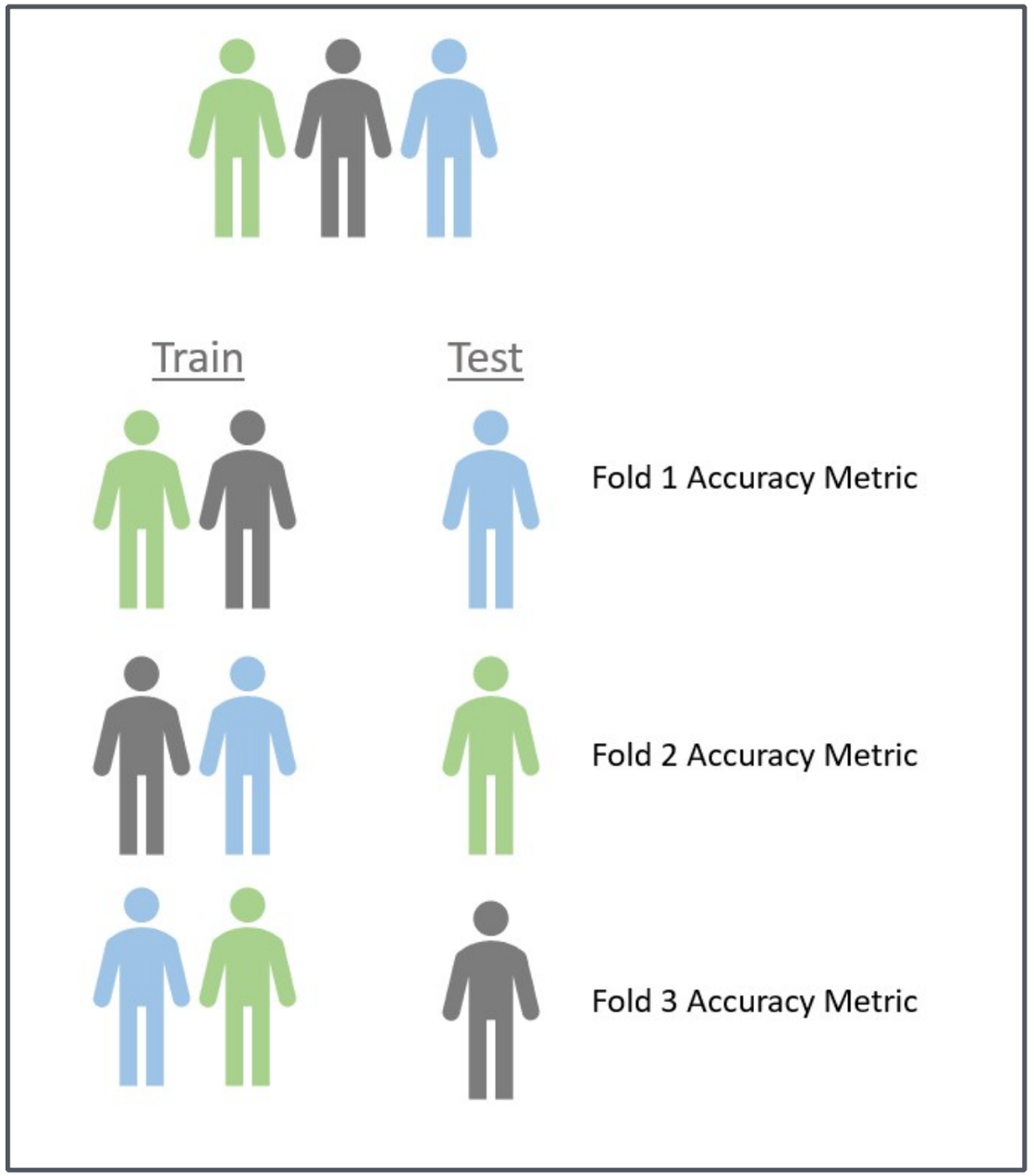
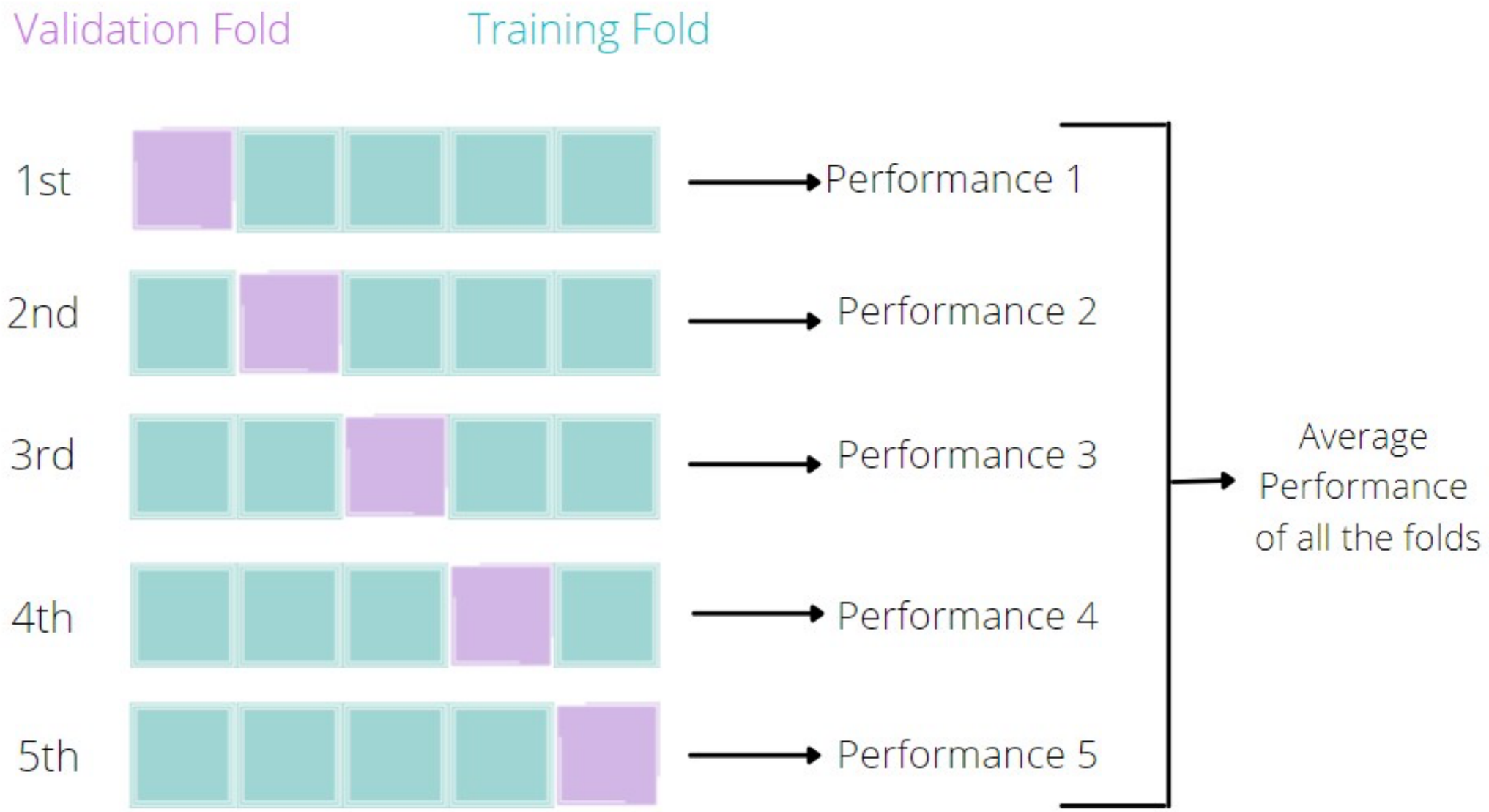


image from: medium.com

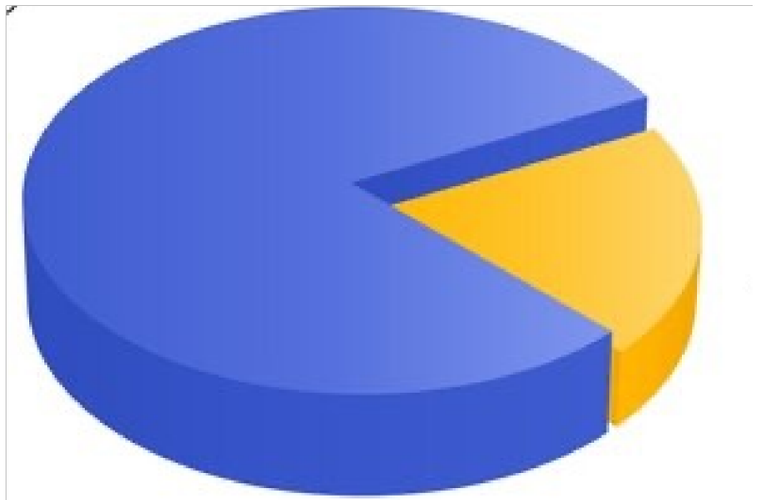
image from: pub.towardsai.net

- **Cross validation**



- **Random train: test splits**
- Particularly useful for very large datasets

- CV may be difficult
- Randomly choose 70-80% data for training and rest for validation testing



Performance metrics

- Performance **metrics** are important to
- Compare performances of multiple classifiers
- Compare performance of the same classifier under different conditions
- Tune hyperparameters

- No metric is perfect; each gives you some insights

- Practical tip: **Use multiple evaluation metrics**

Accuracy and Weighted Accuracy

- **Standard Accuracy:**
  - Ratio of correctly classified samples to the total number of test samples.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Test Samples}}$$

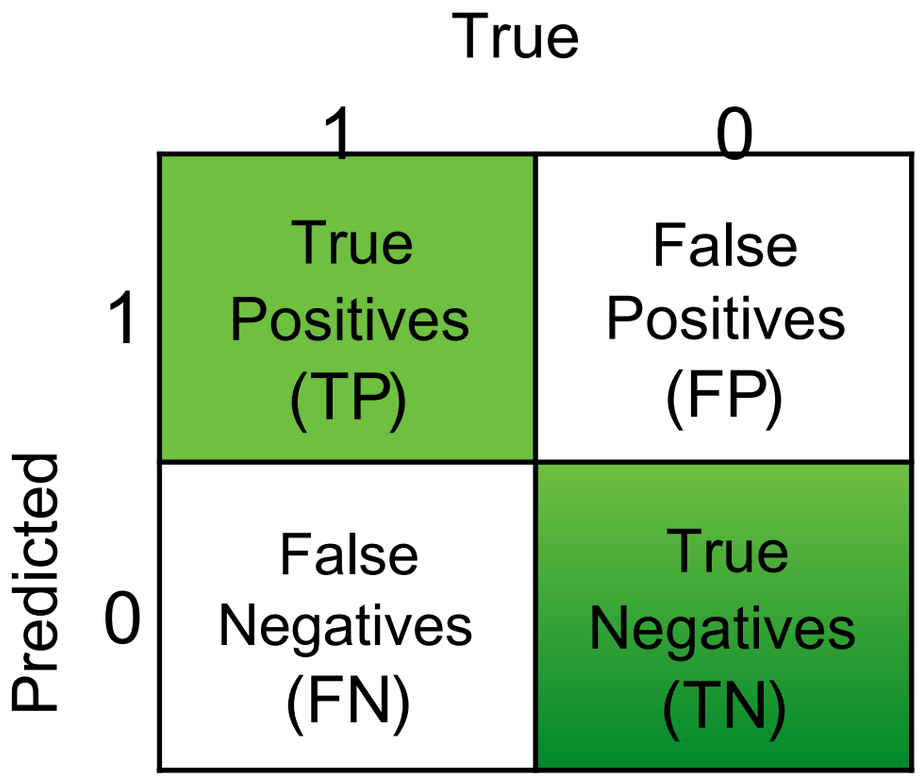
  - Usually expressed as a percentage (%).

- **Weighted Accuracy (WA):**
  - Accuracy is computed independently per class and then averaged across all classes.
  - Helps mitigate the limitations of standard accuracy when dealing with imbalanced datasets.

Accuracy (Failure Example)

- An automated quality-control camera inspects newly manufactured smartphone screens on an assembly line.
- **Data Imbalance:**
  - **99%** of the screens are flawless.
  - **1%** of the screens have a microscopic, fatal crack.
- **"Lazy" Classifier:** A poorly trained inspection system that is too afraid of slowing down the line, so it just stamps **"Pass (Normal)"** on every single screen.
- **Standard Accuracy Trap:**
  - Since 99% of the screens are fine, the system successfully passes 99% of the batch.
  - Its **Standard Accuracy = 99%**.

Confusion matrix



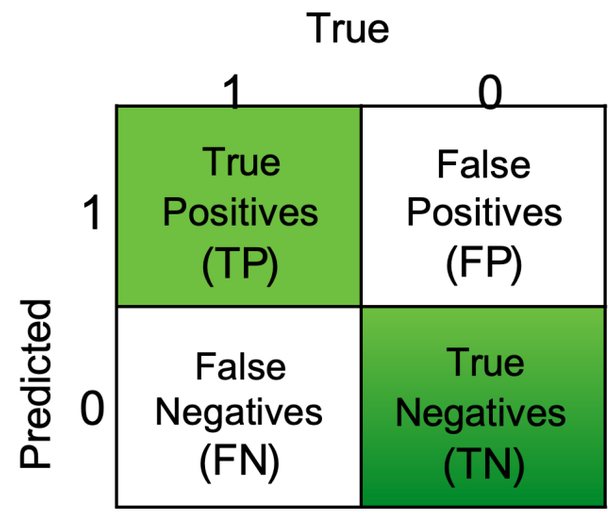
Predicted class \ True class	...	10	11	12	13	14	15	16	18	18	19	20
1	...	4	2	0	2	10	4	7	1	12	7	47
2	...	0	0	4	18	7	8	2	0	1	1	3
3	...	0	0	1	0	1	0	1	0	0	0	0
4	...	1	0	1	28	3	0	0	0	0	0	0
⋮												
16	...	3	2	2	5	17	4	376	3	7	2	**68**
17	...	1	0	9	0	3	1	3	325	3	**95**	19
18	...	2	1	0	2	6	2	1	2	325	4	5
19	...	8	4	8	0	10	21	1	16	19	**185**	7
20	...	0	0	1	0	1	1	2	4	0	1	**92**

- Algorithm is getting 'confused' between classes 20 and 16, and 19 and 17.
  - 17: talk.politics.guns
  - 19: talk.politics.misc
  - 16: talk.religion.misc
  - 20: soc.religion.christian
- Maybe these should be just one class?
- Maybe we need more data in these classes?
- Confusion matrix helps us direct our efforts to improving the classifier.

Precision

- Precision focuses exclusively on the **Predicted Positive** row (Predicted = 1).
- Among all instances classified as positive (e.g., "diseased"), how many are actually positive?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$



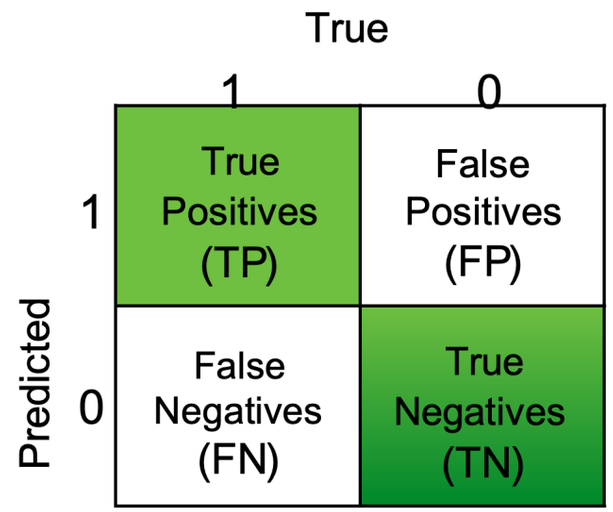
- Perfect precision means zero False Positives (FP = 0), meaning every positive prediction is correct.
- Higher precision is always better.

When to use: Crucial when the cost of a **False Positive** is very high (e.g., falsely accusing an innocent person).

Recall or Sensitivity

- *Among all actual positive instances (e.g., "diseased" patients), how many were correctly identified by the model?*

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN}$$

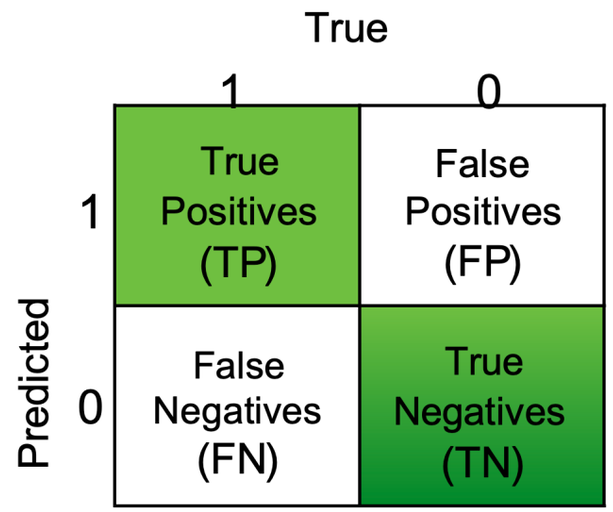


- Perfect recall means zero False Negatives ($FN = 0$), meaning the model missed zero positive cases.
- Higher recall is better.
- **When to use:** Crucial when the cost of a **False Negative** is dangerously high (e.g., failing to diagnose a serious disease or missing a security threat).

Specificity

- Specificity focuses exclusively on the **Actual Negative** column (True $= 0$).
- Among all actually healthy people, how many were correctly classified as healthy?

$$\text{Specificity } (S_p) = \frac{\text{TN}}{\text{TN} + \text{FP}}$$



- Perfect specificity means zero False Positives (FP $= 0$), meaning no healthy individuals are misdiagnosed.
- Higher specificity is better.
- **When to use:** Essential when the cost of a **False Positive** (false alarm) is high, ensuring healthy cases are correctly protected from unnecessary panic or intervention.

Optimizing sensitivity and specificity

- We would like both to be as high as possible.
- Often increasing one will decrease the other.
- Balance will depend on application:
- e.g. diagnosis:
  - We can probably tolerate a decrease in specificity (healthy people diagnosed as diseased)....
  - ...if it gives us an increase in sensitivity (getting diseased people right).

F1 Score

- A single metric combining both **Precision** and **Recall** into a balanced score.

- Calculated as the harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

- **Key Characteristics:**
  - Bounded between 0 and 1.
  - Higher values indicate better model performance.

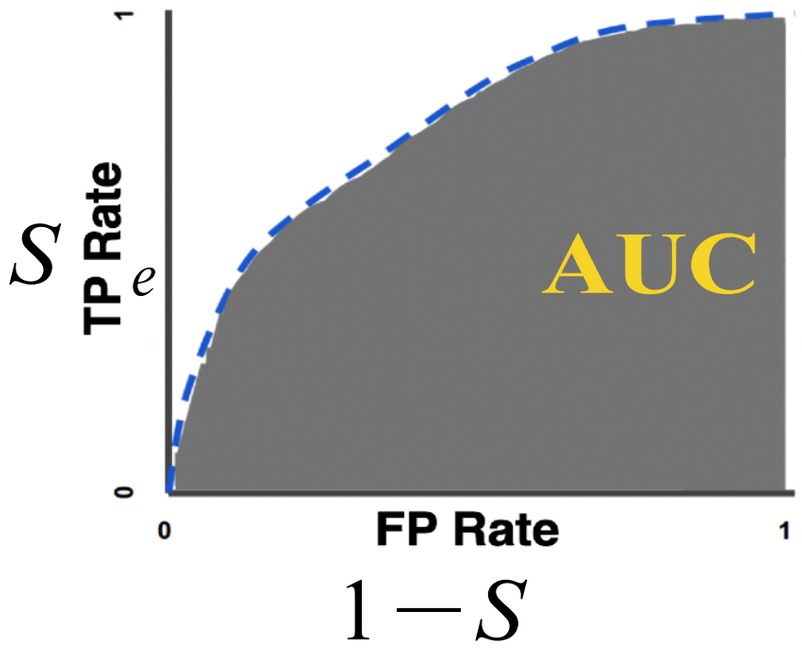
ROC

- Many classification algorithms involve setting a threshold.
- e.g. Logistic Regression:

$$p(t_{new} = 1 | \mathbf{x}_{new}, \mathbf{w}) > 0.5$$

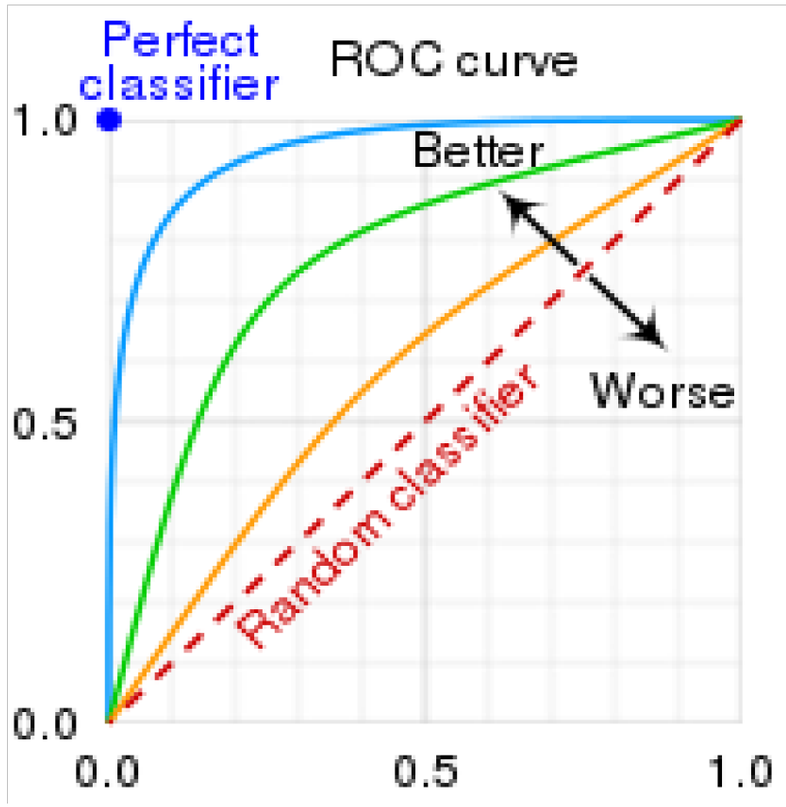
- Implies a threshold of zero (sign function)
- However, we could use any threshold we like....
- The *Receiver Operating Characteristic (ROC) curve* shows how S_e and $1 - S_p$ vary as the threshold changes.

ROC, AUC



S_e

$1-S_p$



S_e

$1-S_p$

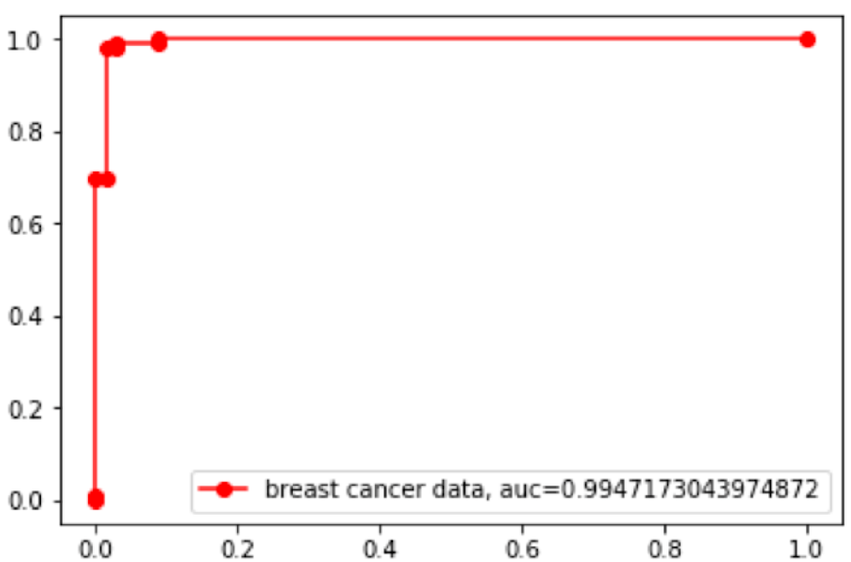
[Image from Wikipedia]

```
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.datasets import load_breast_cancer

breast_cancer = load_breast_cancer()
X = breast_cancer.data
t = breast_cancer.target


X_train, X_test, y_train, y_test = train_test_split(X,t,test_size=0.30, random_sta
te=123)
clf1 = LogisticRegression().fit(X_train, y_train)

y_pred1 = clf1.predict(X_test)
y_pred_proba1 = clf1.predict_proba(X_test)[:,1]
fpr1, tpr1, _ = metrics.roc_curve(y_test,  y_pred_proba1)
auc1 = metrics.roc_auc_score(y_test, y_pred_proba1)
plt.plot(fpr1,tpr1,'ro-',label="breast cancer data, auc="+str(auc1))
plt.legend(loc=4)
plt.show()
```

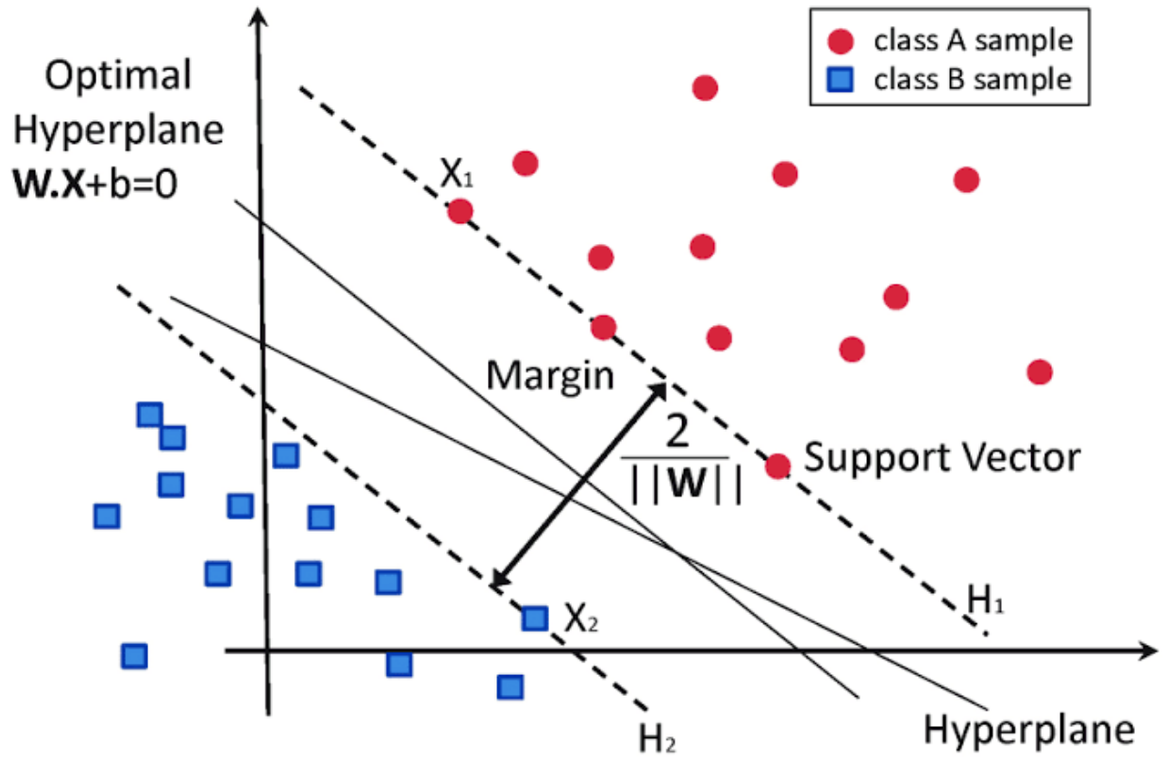


Try it on a breast cancer dataset
Plot ROC of a Logistic Regression

Classification: Support Vector Machine

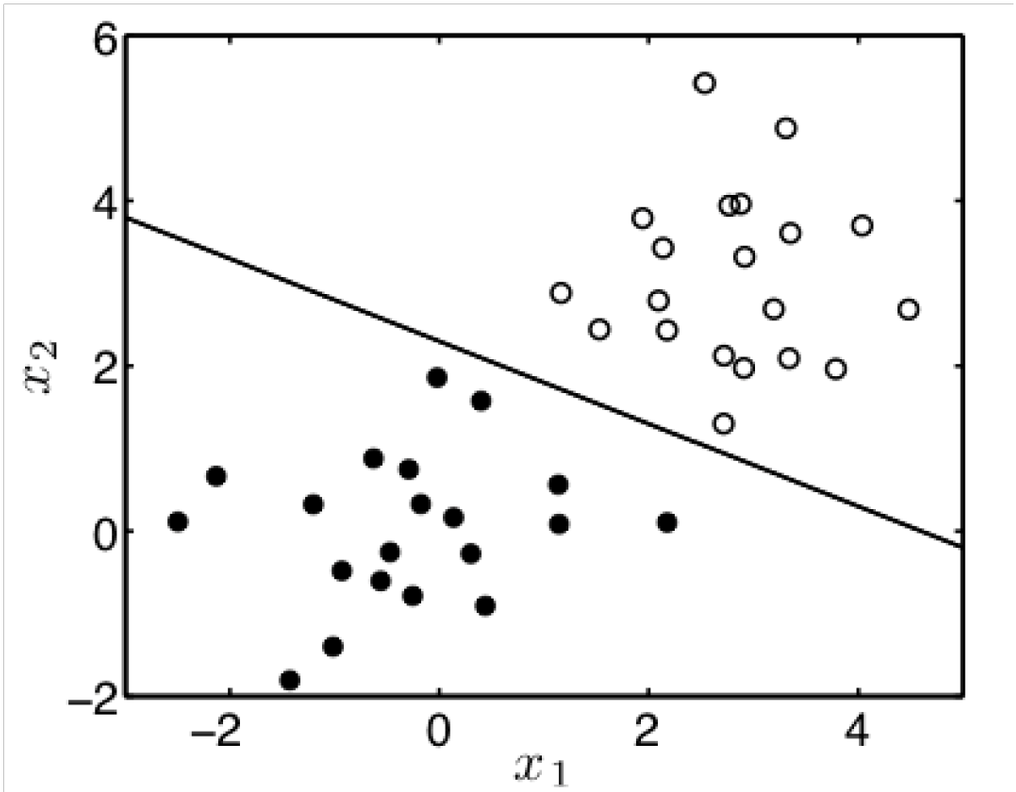
Support Vector Machine (SVM)

- Classifiers are learned by minimizing a loss or maximising likelihood
- SVM is no different:
    - Finds a *hyperplane* (decision boundary) in an n-dimensional space separating data points to classes
    - The hyperplane should maximise the distance from data points of either classes



- SVM is a non-probabilistic (binary) classifier

SVM



- N data points, each with feature vector:

$\mathbf{x} = [x_1, x_2]^T$

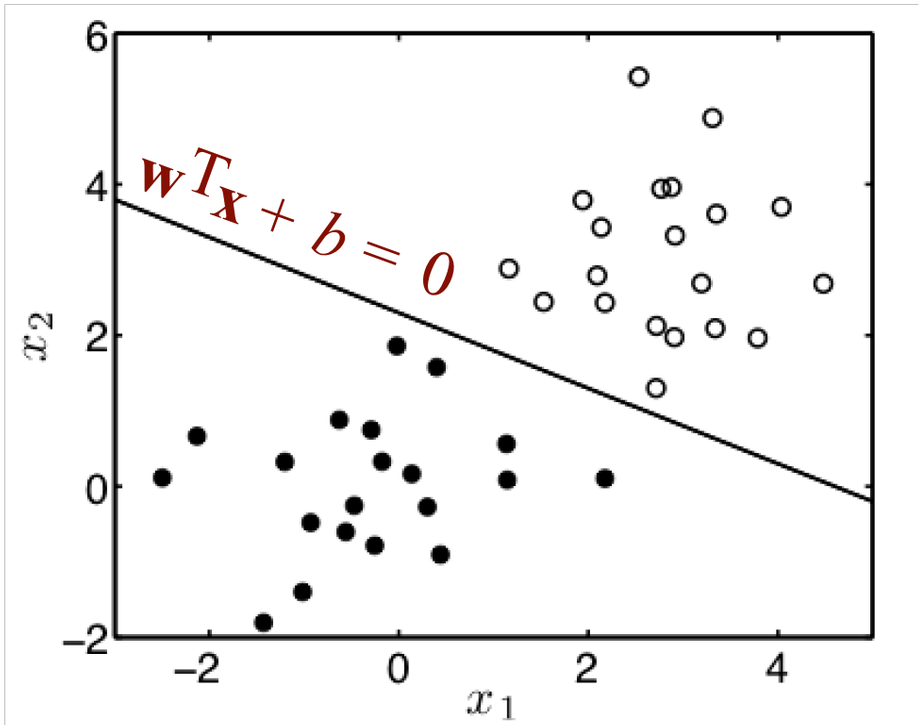
- Labels: $y \in \{1, -1\}$

- A linear decision boundary can be represented as a straight line

$$\mathbf{w}^T\mathbf{x} + b = 0$$

- Given training data, our task is to find $\mathbf{w}$ and b

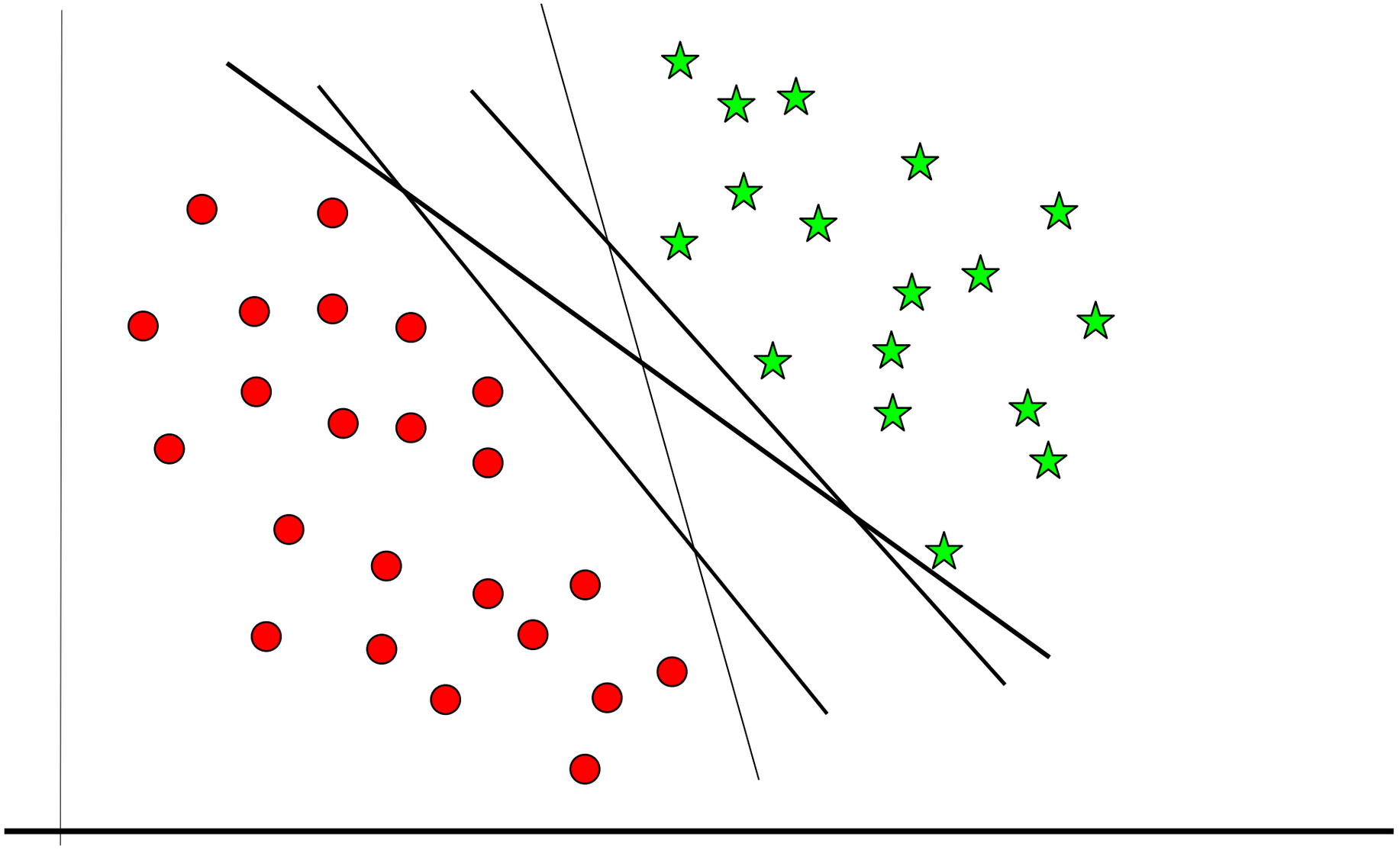
SVM



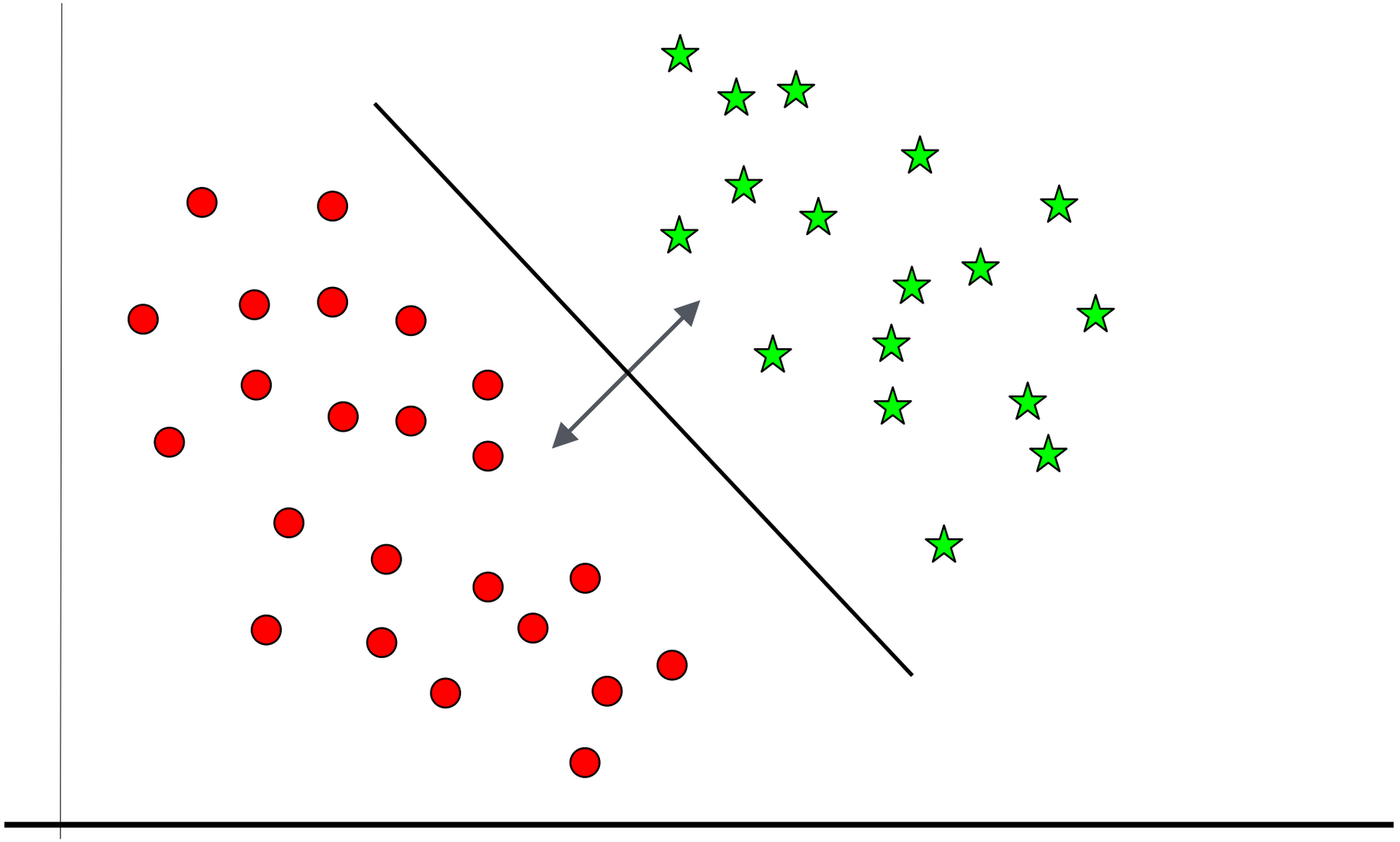
- N data points, each with feature vector: $\mathbf{x} = [x_1, x_2]^T$
- Labels: $y \in \{1, -1\}$

- Classification: $\mathbf{w}^T \mathbf{x}_{new} + b > 0 \quad : \quad y_{new} = 1 \; \mathbf{w}^T \mathbf{x}_{new} + b < 0 \quad : \quad y_{new} = -1$
- Classification: $y_{new} = \text{sign}(\mathbf{w}^T \mathbf{x}_{new} + b)$

There might be more than one possible classifiers for this dataset. Are they all equally-good?



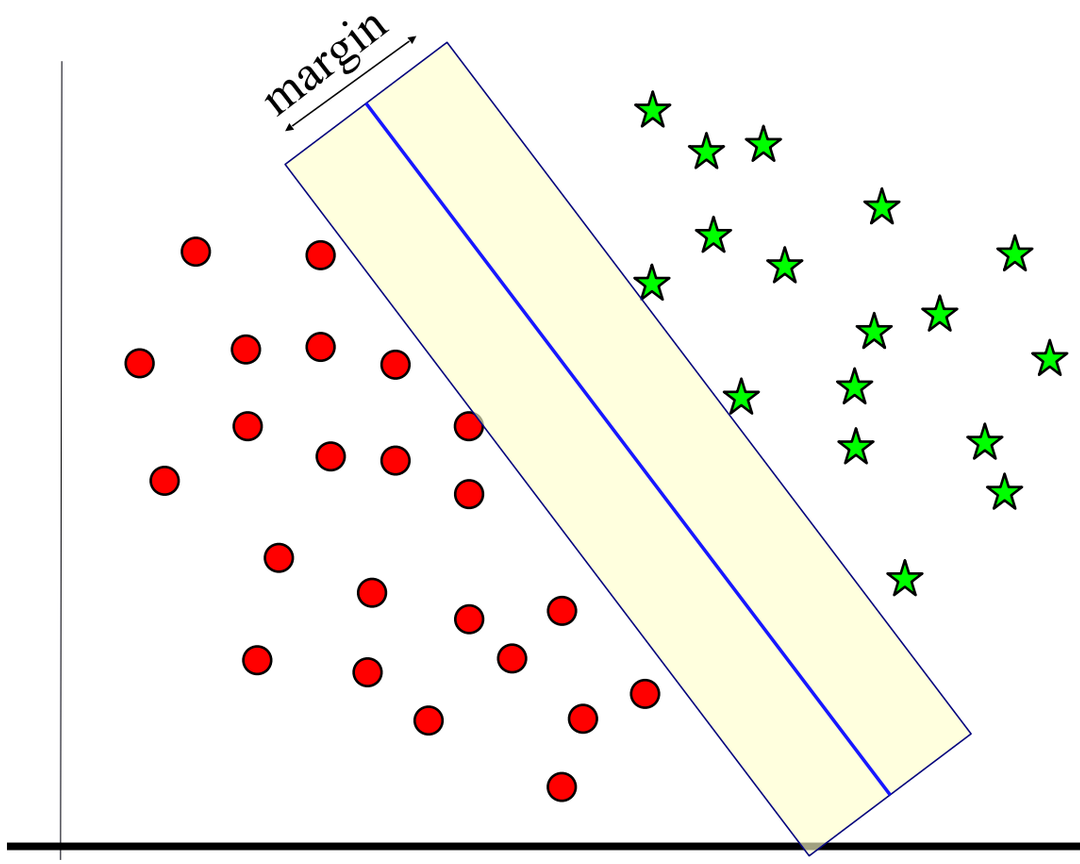
A hyperplane that separates two classes and maximizes the distance to the closest point from either class, i.e., **maximizes the margin** of the classifier.



Ensuring that a classifier is *not too close* to any data point leads to better generalization:

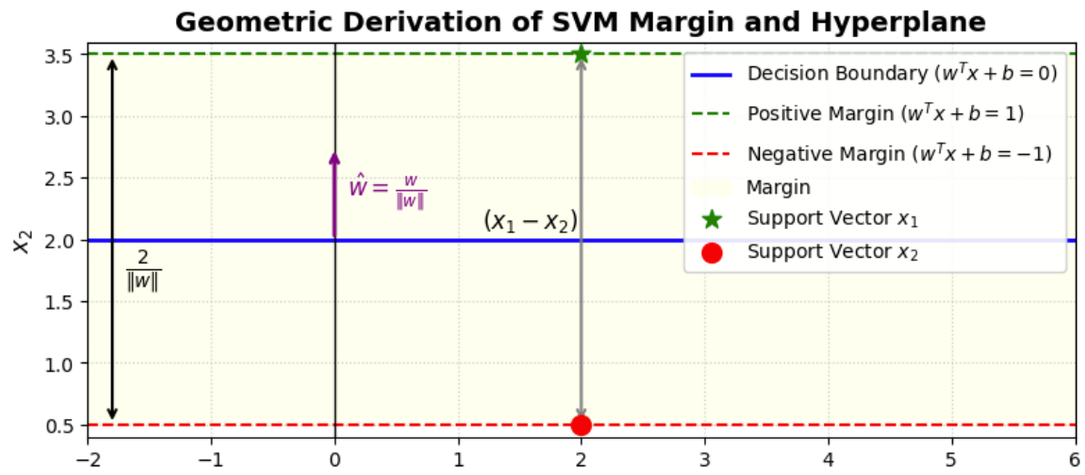
- if $\mathbf{x}_{new}$ is close to its training samples, then our classifier performs well

before making an error



- if $\mathbf{x}_{new}$ is not close to the training data, the margin leaves more room

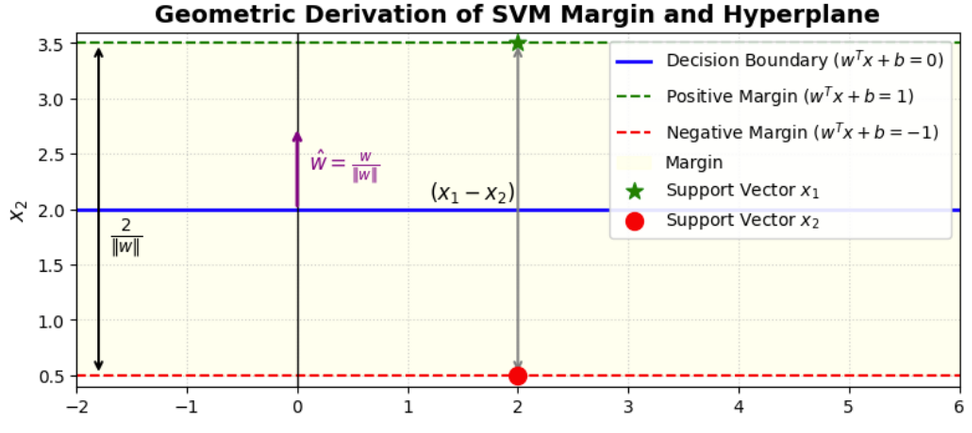
SVM – Distance to Decision Hyperplane



Component	Mathematical Expression / Formula	Description
Hyperplane Normal	$\mathbf{w}$	The weight vector $\mathbf{w}$ is strictly orthogonal (perpendicular) to the decision hyperplane.
Unit Normal Vector	$\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$	The normalized unit vector pointing in the direction of $\mathbf{w}$.
Point Difference	$(\mathbf{x}_1 - \mathbf{x}_2)$	The vector connecting two points $\mathbf{x}_1$ and $\mathbf{x}_2$.
Projection Distance (d)	$d = \frac{1}{\|\mathbf{w}\|}\mathbf{w}^T(\mathbf{x}_1 - \mathbf{x}_2)$	The signed orthogonal distance, calculated as the scalar projection of $(\mathbf{x}_1 - \mathbf{x}_2)$ onto the unit normal vector $\hat{\mathbf{w}}$.

SVM: From Geometric Margin to Optimization

- **Maximize the Margin:**
  - Maximize distance $d = \frac{1}{\|\mathbf{w}\|}$.



- **Simplify the Objective:**
  - Maximizing $\frac{1}{\|\mathbf{w}\|}$ is equivalent to **minimizing** $\|\mathbf{w}\|$, which is mathematically smoother as minimizing $\frac{1}{2}\|\mathbf{w}\|^2$.
- **Establish Constraints:**
  - For any training point $\mathbf{x}_i$, we require correct classification outside the margin:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad \forall i$$

- **Optimization Problem:**

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \forall i$$

SVM Loss Formulations

- **Primal Optimization Problem (Hard Margin):** Expressed as minimizing the weight norm:

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \text{ subject to the constraints } y_i \mathbf{w}^T \mathbf{x}_i \geq 1 \text{ for all } i.$$

- **Lagrangian Form:** Can be reformulated using Lagrange multipliers (α_i) to handle the constraints:

$$\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i \left(y_i \mathbf{w}^T \mathbf{x}_i - 1 \right) \right) \text{ subject to } \alpha_i \geq 0.$$

- **L2-Regularised Hinge Loss (Most Commonly Used in Practice):**
  - Written in an unconstrained format as:

$$\min_{\mathbf{w}} \sum_i \max(0, 1 - y_i \mathbf{w}^T \mathbf{x}_i) + \frac{\lambda}{2}\|\mathbf{w}\|^2.$$

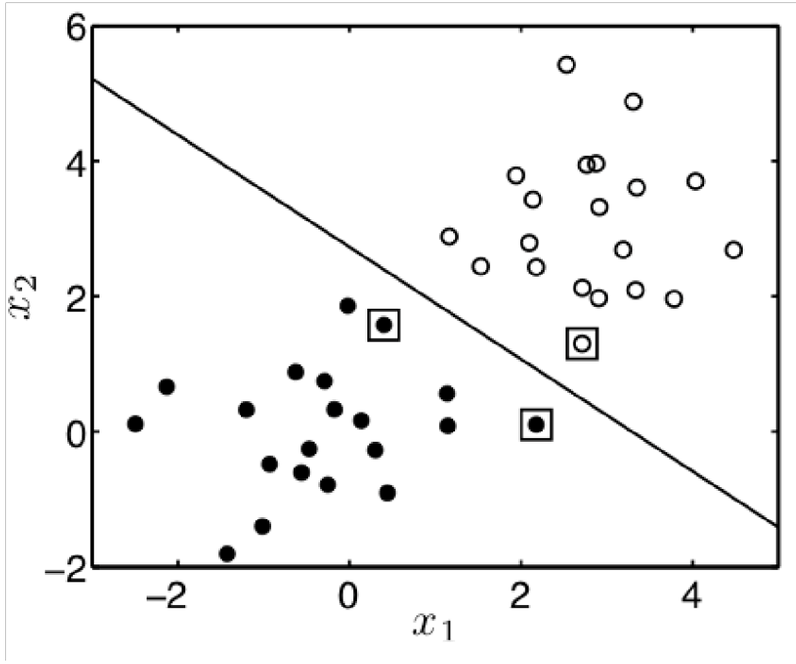
  - This formulation penalizes margin violations while regularizing the model and can be solved efficiently using standard gradient-based optimization solvers.

Support vectors

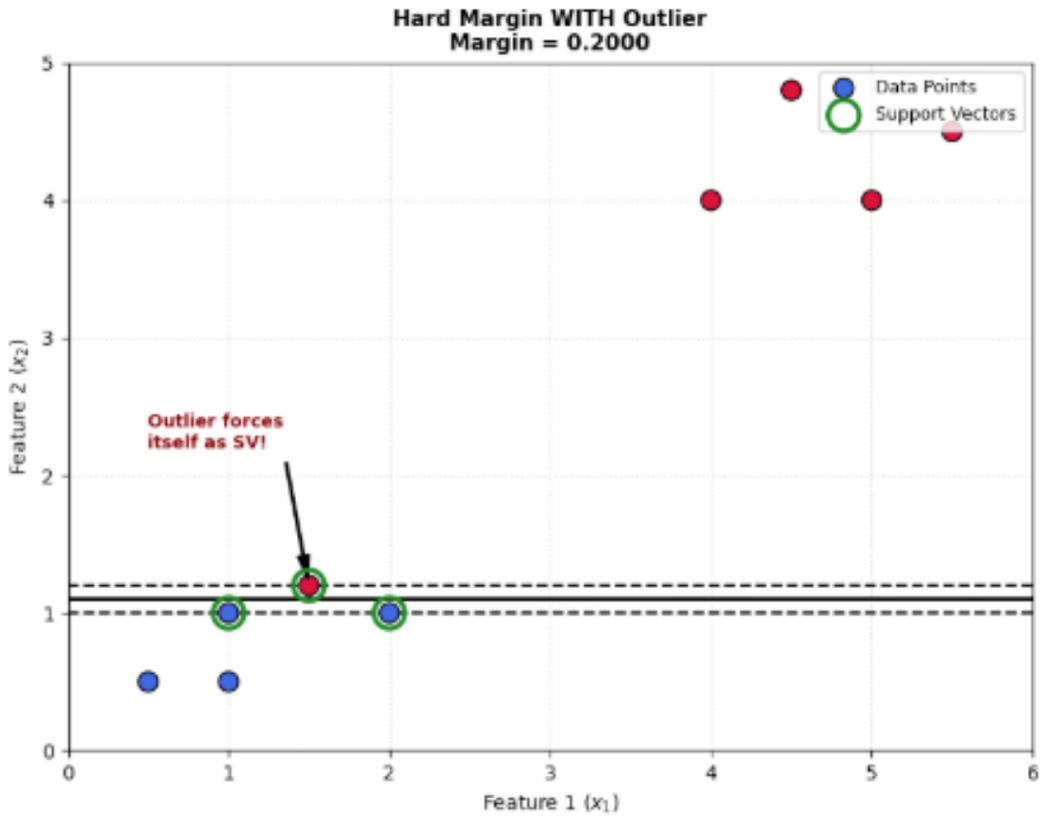
- Prediction of hyperplane only depends on a small number of data points.
- A small number of non-zero weights

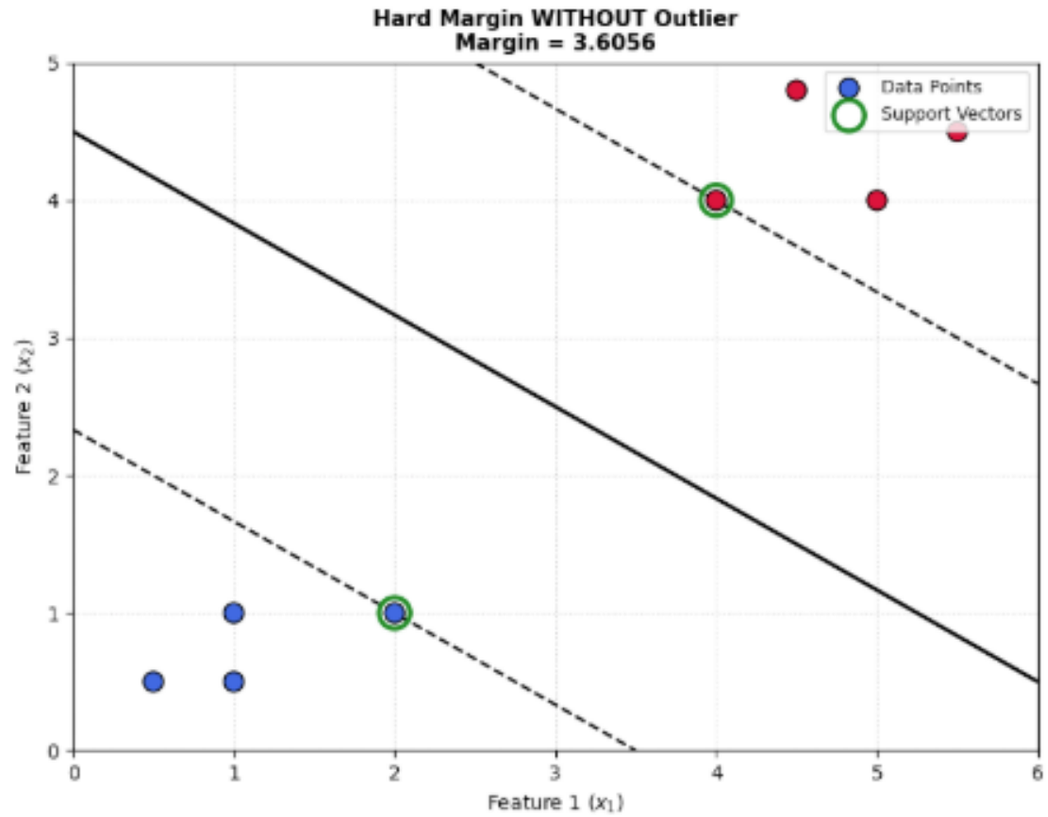
- These are called **support vectors**

- The constraints we used requires all training samples to be correctly classified - this creates a **hard margin**



Support vectors: Hard vs Soft Margin

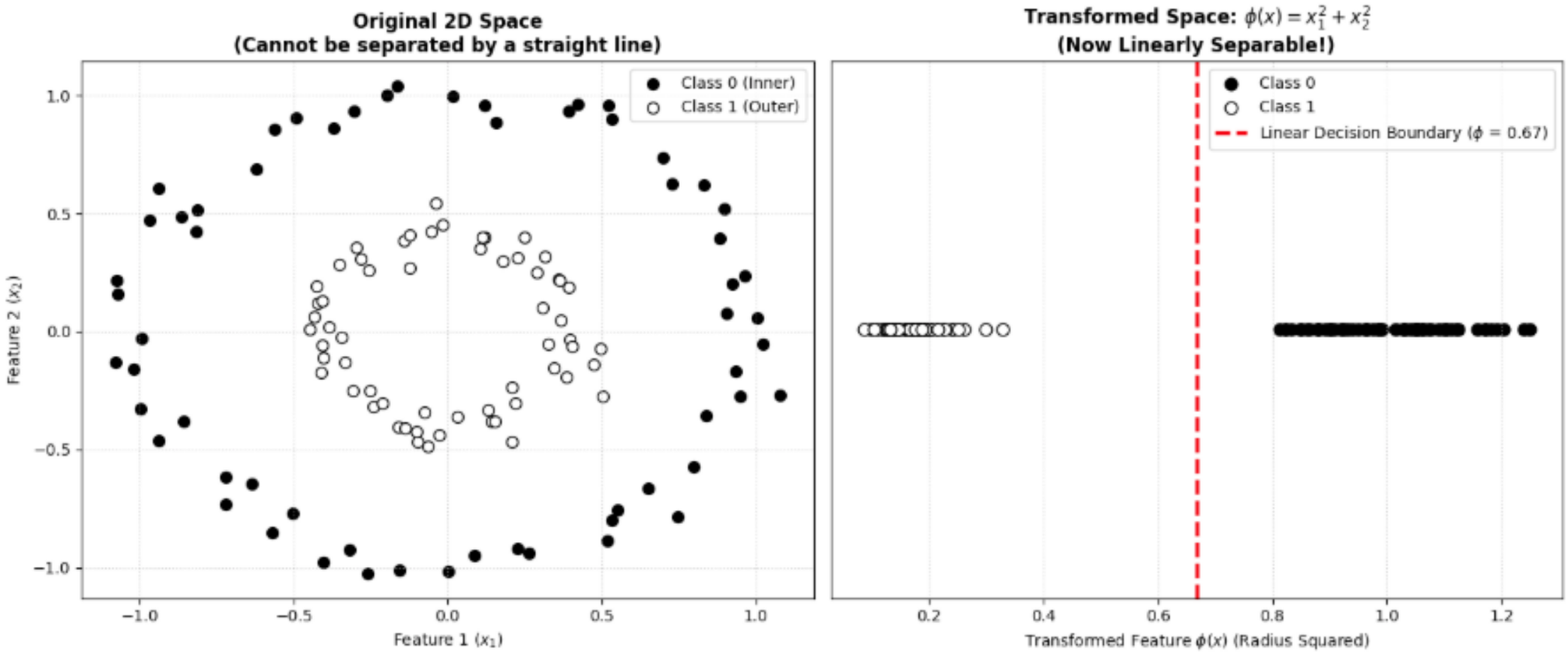




SVM – Handling Non-linear Data (Kernel trick)

- **The Limitation:**
  - So far, we have learned linear classifiers. What if the dataset is **not linearly separable**?
- **Feature Space Transformation:**
  - Introduce non-linearity by mapping input features into a higher-dimensional space using a transformation function $\phi(x)$.
- **The Objective:**
  - Design the transformation so that the data **becomes linearly separable** in this new feature space.
- **Example Kernel Function:**
  - $k(x) = x_1^2 + \cdots + x_D^2$

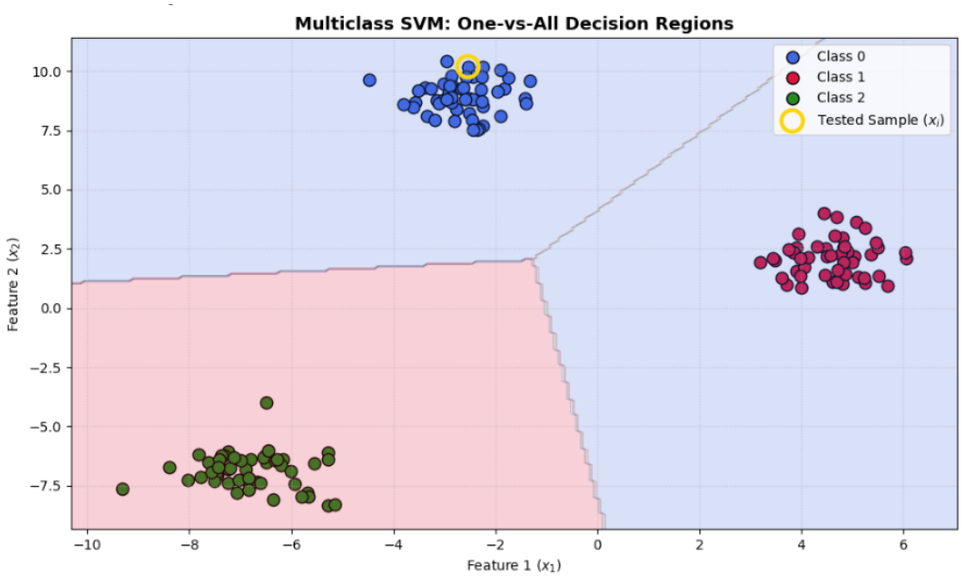
Kernel trick



Multiclass SVM

One-vs-All Strategy:

- Train one binary classifier for each of the C classes.
- Treat samples of target class c as *positive* and all other classes as *negative*.



- **Scoring & Weight Matrix (W):**
  - For each class c, compute the discriminant score $w_c^T x_i$.
  - Construct a weight matrix W where the rows consist of weight vectors $w_1, \dots, w_C$.
- **Prediction Vector (z_i):**
  - For a data point belonging to class 1, the target prediction vector takes the form $z_i = [1, -1, \dots, -1]^T$.
  - Expressed compactly in matrix notation as $z_i = W x_i$.

Multiclass SVM Formulation & Loss

- **Scoring & Matrix Notation:**
  - For each class, we compute the discriminant score as $w_c^T x_i$.
  - For a data point belonging to class y_i, the target prediction vector is $z_i = [1, -1, \ldots, -1]^T$.
  - This is expressed compactly as $z_i = W x_i$, where W has weight vectors $w_1, \ldots, w_C$ as its rows.
- **Multiclass SVM Loss Function:**
  - To train the weight matrix W, we minimize the regularized margin loss across all training samples:

$$\min_W \sum_i \max_{c \neq y_i} \left\{ \max\left(0, 1 - w_{y_i}^T x_i + w_c^T x_i\right) \right\} + \frac{\lambda}{2} \|W\|_F^2$$

  - **Margin Violation Term:** Ensures the true class score ($w_{y_i}^T x_i$) exceeds any incorrect class score ($w_c^T x_i$) by at least a margin of 1.
  - **Frobenius Norm Regularisation ($\|W\|_F^2$):** Prevents overfitting and controls model complexity.

Summary

- SVM is a *maximum-margin classifier* for 2 classes: choose the farthest hyperplane from both classes
- Parameter λ controls trade-off between having a large margin and classifying examples correctly
- Powerful classifier, can model complex data
- Kernel trick is very useful
- Can be extended to multi class
- Was state-of-the-art before deep neural networks